

STATS 786: Group Project

Average Monthly Temperatures in Wellington

Group Leader: [palceholder]

Group Members: [Eric Heo], [Dominicus Nararya], [Simon Loh]

June 2, 2026

Contents

1	Data Cleaning and Exploratory Data Analysis	2
1.1	Visualizing the Time Series	5
2	ETS Models	8
3	ARIMA Models	10
3.1	Transformations/ Differencing Required?	10
3.2	Methodology	11
3.3	Backshift Notation	13
4	Dynamic Regression Models	13
4.1	Dynamic Regression Explanation	13
4.2	Model Exploration and Selection	14
5	Assumption Checking	15
5.1	ETS Models Assumptions	15
5.2	ARIMA Models Assumptions	17
5.3	Dynamic Regression Models Assumptions	17
6	Forecasting	19
6.1	ETS Models Forecasting	19
6.2	ARIMA Models Forecasting	20
6.3	Dynamic Regression Models Forecasting	23
6.4	Forecast Comparison	24
7	LLMs in Time Series Analysis	27
8	Member Contributions	27

1 Data Cleaning and Exploratory Data Analysis

The dataset contains average monthly temperatures for Wellington. To prepare the data for modeling, we first formatted the date index and cast the data into a `tsibble` object. As per the project requirements, we split the data into a training set (all data up to April 2025) and a test set containing the final $h = 12$ months (May 2025 to April 2026) to evaluate our models' true predictive performance.

We also scanned the series for missing values. Because temperature data is heavily driven by the time of year, simple imputation methods like Last Observation Carried Forward (LOCF) or linear interpolation can distort the natural seasonal curve if a gap occurs during a seasonal peak or trough. To determine the best strategy, we plotted multiple imputation techniques against each other. We selected Seasonal Interpolation (`na_seadec`), which decomposes the series, interpolates the seasonally adjusted data, and adds the seasonality back in, preserving the natural climate cycles.

```
data <- read_csv("wellington.csv") %>%
  mutate(Date = yearmonth(paste(Year, Month))) %>%
  select(-Month, -Year) %>%
  as_tsibble(index = Date)

# scanning for missing values
colSums(is.na(data))

## Temperature      Date
##              0      0

scan_gaps(data)

## # A tsibble: 15 x 1 [1M]
##       Date
##       <mth>
## 1 1993 May
## 2 1993 Jun
## 3 1993 Jul
## 4 1993 Aug
## 5 1993 Sep
## 6 1993 Oct
## 7 1993 Nov
## 8 1993 Dec
## 9 1994 Jan
## 10 1994 Feb
## 11 1994 Mar
## 12 1994 Apr
## 13 1994 May
## 14 1994 Jun
## 15 1994 Jul

# filling implicit gaps in time index
data <- data %>% fill_gaps()

# convert to ts for imputeTS methods
temp_ts <- ts(data$Temperature, frequency = 12)
```

```

# create comparison dataframe with all methods
comparison <- data %>%
  as_tibble() %>%
  mutate(
    Original      = Temperature,
    LOCF          = as.numeric(na.locf(temp_ts, na.rm = FALSE)),
    NOCB          = as.numeric(na.locf(temp_ts, fromLast = TRUE, na.rm = FALSE)),
    Linear        = as.numeric(na_interpolation(temp_ts, option = "linear")),
    Spline        = as.numeric(na_interpolation(temp_ts, option = "spline")),
    Seasonal      = as.numeric(na_seadec(temp_ts, algorithm = "interpolation"))
  ) %>%
  select(Date, Original, LOCF, NOCB, Linear, Spline, Seasonal) %>%
  pivot_longer(-Date, names_to = "Method", values_to = "Temperature")

comparison %>%
  mutate(Method = factor(Method, levels = c("Original", "LOCF", "NOCB",
                                             "Linear", "Spline", "Seasonal"))) %>%

  ggplot(aes(x = Date, y = Temperature)) +
  geom_line(linewidth = 0.3) +
  facet_wrap(~ Method, ncol = 1) +
  labs(title = "Wellington Monthly Temperature | Interpolation Method Comparison",
       y = "Temperature (°C)", x = "Date") +
  theme_minimal() +
  theme(strip.text = element_text(face = "bold"))

```

Wellington Monthly Temperature – Interpolation Method Comparison

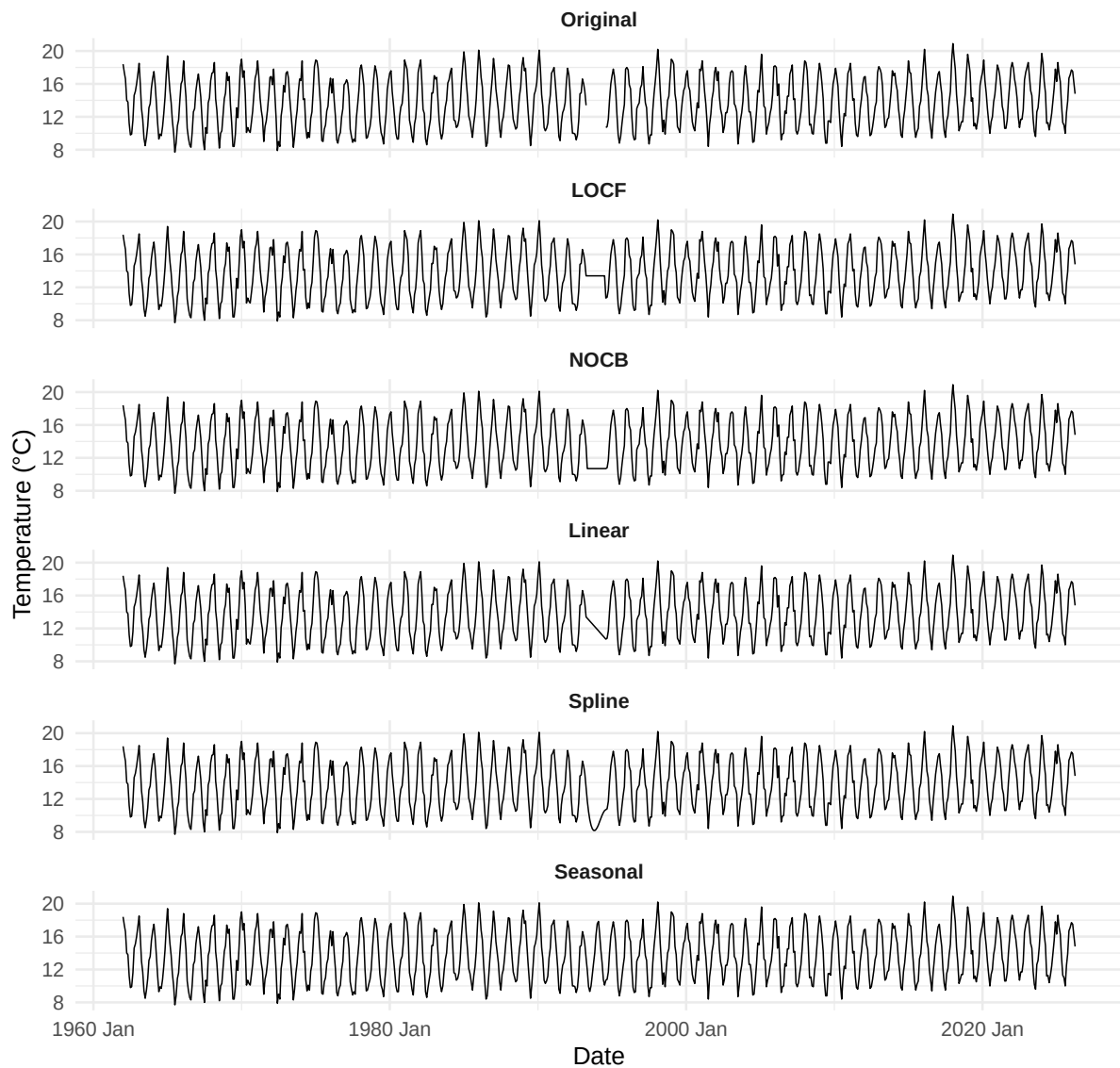


Figure 1: Comparison of Missing Value Imputation Methods

```
# apply chosen seasonal imputation
data <- data %>%
  mutate(Temperature = na_seadec(Temperature, algorithm = "interpolation", frequency(12))) %>%
  mutate(Temperature = as.numeric(Temperature))

# creating training + test sets
training <- data %>%
  filter(Date <= yearmonth("2025 April"))

test <- data %>%
  filter(Date > yearmonth("2025 April"))
```

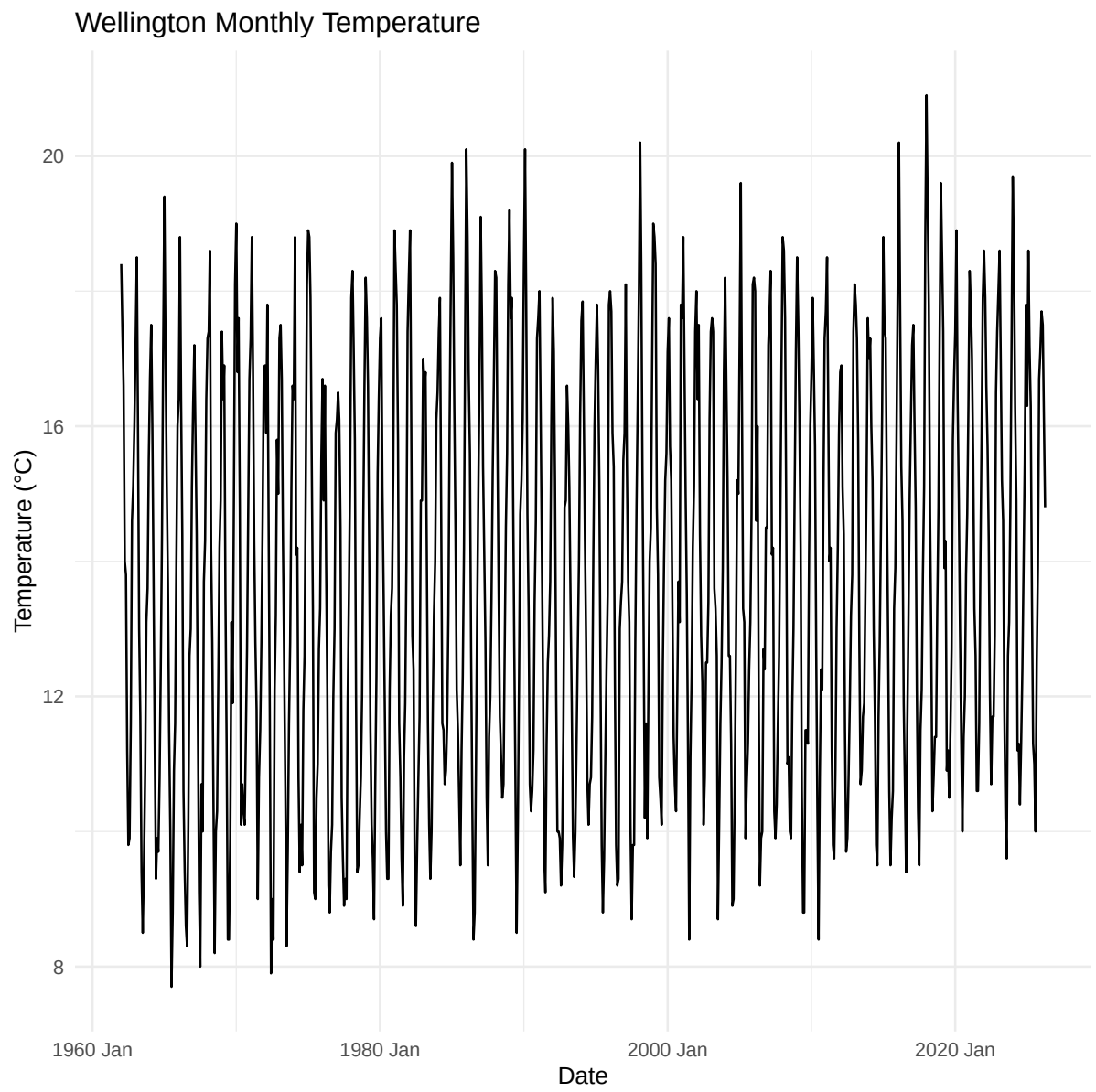
1.1 Visualizing the Time Series

To understand the underlying behavior of Wellington's temperatures, we produced a standard time plot, a seasonal subseries plot, and an STL decomposition.

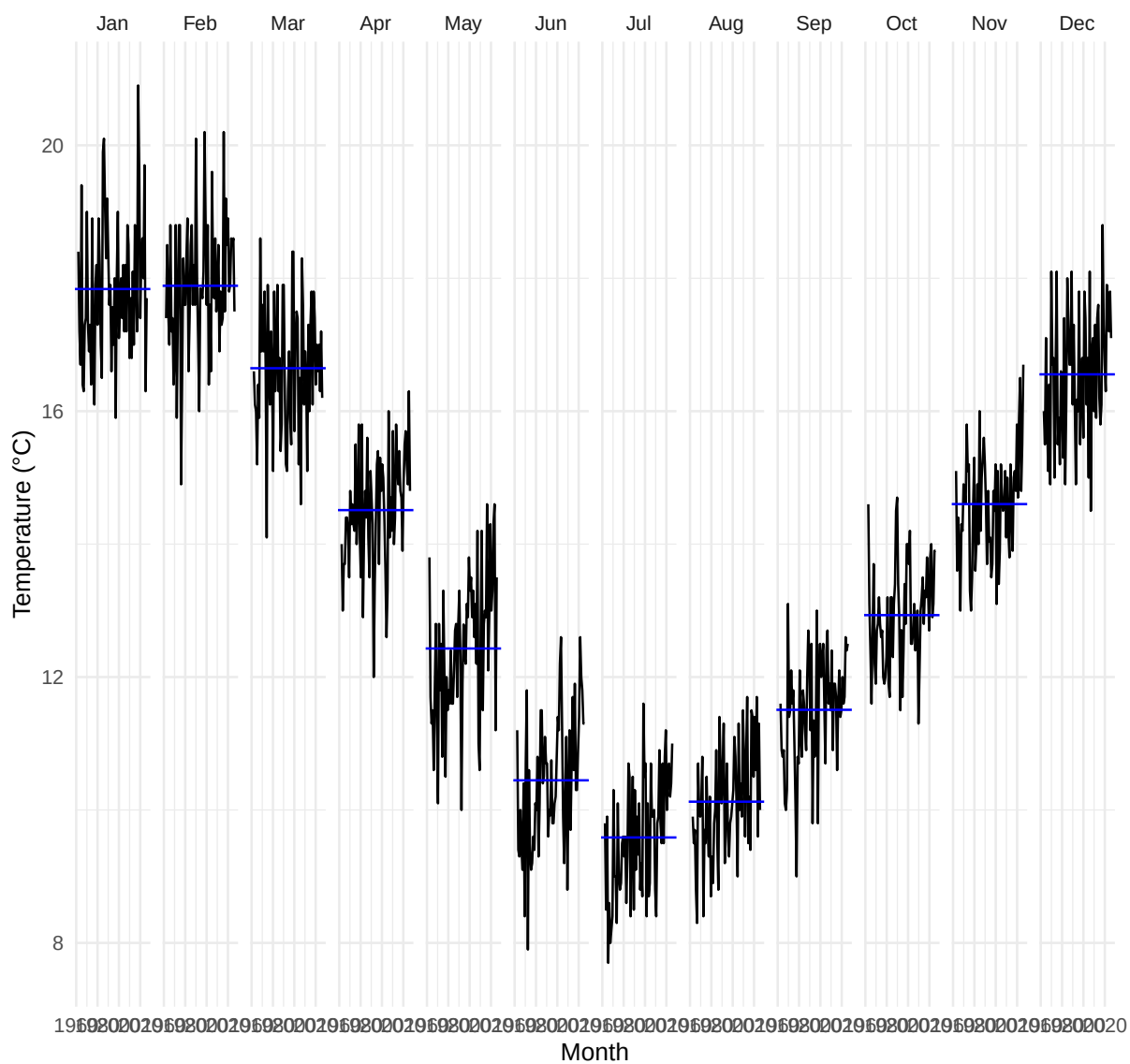
```
# standard time plot
data %>%
  autoplot(Temperature) +
  labs(title = "Wellington Monthly Temperature",
        y = "Temperature (°C)",
        x = "Date") +
  theme_minimal()

# seasonal subseries plot
data %>%
  gg_subseries(Temperature) +
  labs(title = "Wellington Temperature Seasonal Subseries",
        y = "Temperature (°C)",
        x = "Month") +
  theme_minimal()

# STL decomp
data %>%
  model(STL(Temperature ~ trend() + season(window = "periodic"))) %>%
  components() %>%
  autoplot() +
  labs(title = "STL Decomposition of Wellington Temperature") +
  theme_minimal()
```

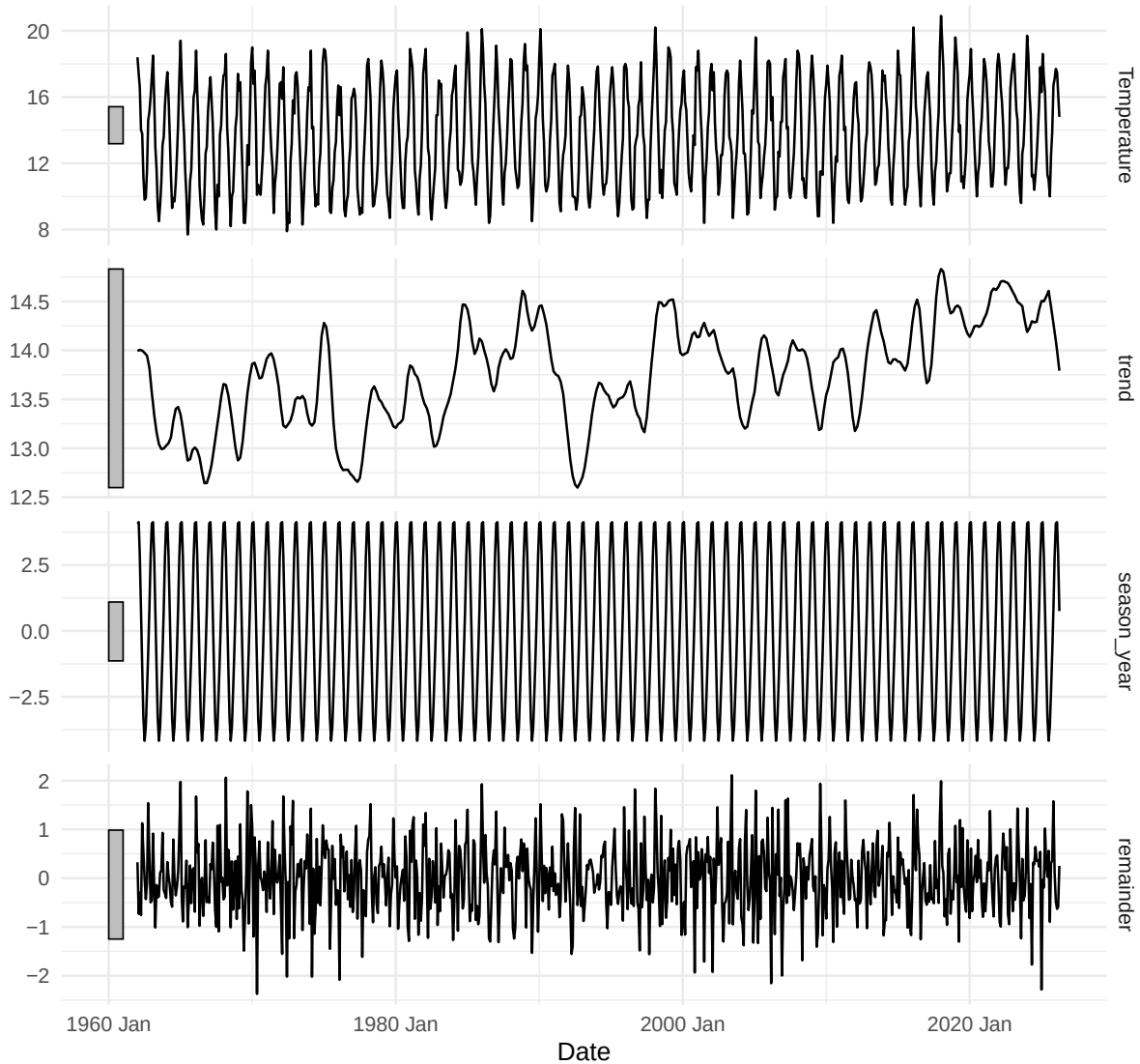


Wellington Temperature Seasonal Subseries



STL Decomposition of Wellington Temperature

Temperature = trend + season_year + remainder



Observed Features: palceholder ghaksfsgjdklnbkjdnkjsjkgserf

2 ETS Models

The trend component in the STL decomposition shows that there is a slight upward trend with Temperature increasing over time with peaks and troughs of 14.0 and 12.5 C in the 1960's to 15.0 and 13.5 C in the 2010's. Three trend specifications were considered no trend (N), additive trend (A), and damped additive trend (Ad).

The remainder component shows roughly constant variance throughout the whole time series from 1960 to 2025 meaning there is no heteroscedacity. This means that additive errors (A) are appropriate, so we will not include multiplicative error models.

The seasonal component shows highly stable yearly annual cycle that remains unchanged over the whole time series. As the magnitude of seasonal fluctuations does not vary, additive seasonality (A) is appropriate. We will not consider multiplicative seasonality.

As such the following models were shortlisted: ETS(A,N,A), ETS(A,A,A), and ETS(A,Ad,A).


```

fit <- training %>%
  model(
    ANA = ETS(Temperature ~ error("A") + trend("N") + season("A")),
    AAA = ETS(Temperature ~ error("A") + trend("A") + season("A")),
    AAdA = ETS(Temperature ~ error("A") + trend("Ad") + season("A")),
    auto = ETS(Temperature)
  )

fit %>% select(auto) %>% report()

## Series: Temperature
## Model: ETS(A,N,A)
## Smoothing parameters:
##   alpha = 0.125848
##   gamma = 0.0001000595
##
## Initial states:
##   l[0]    s[0]    s[-1]    s[-2]    s[-3]    s[-4]    s[-5]    s[-6]
## 13.75272 2.827684 0.8530713 -0.8283216 -2.208586 -3.629442 -4.199804 -3.342363
##   s[-7]    s[-8]    s[-9]    s[-10]    s[-11]
## -1.297429 0.7641372 2.861139 4.11656 4.083353
##
## sigma^2: 0.7357
##
##      AIC      AICc      BIC
## 4823.964 4824.610 4893.464

glance(fit) %>%
  select(.model, AIC, AICc, BIC) %>%
  arrange(AICc)

## # A tibble: 4 x 4
##   .model  AIC  AICc  BIC
##   <chr>  <dbl> <dbl> <dbl>
## 1 ANA    4824. 4825. 4893.
## 2 auto   4824. 4825. 4893.
## 3 AAdA   4830. 4831. 4914.
## 4 AAA    4834. 4835. 4912.

```

The ETS(A,N,A) model will likely have the best predictive ability as it has the lowest AICc (manually) and was also the best model automatically selected. This suggests that even though there is a slight increasing trend of temperature in the STL decomposition, the trend component does not improve out-of-sample predictive ability.

The fitted ETS(A,N,A) model equations are as follows:

$$y_t = \ell_{t-1} + s_{t-12} + \varepsilon_t$$

$$\ell_t = \ell_{t-1} + 0.1258\varepsilon_t$$

$$s_t = s_{t-12} + 0.0001\varepsilon_t$$

where $\varepsilon_t \sim \text{NID}(0, 0.7357)$ and $\ell_0 = 13.7527$.

3 ARIMA Models

3.1 Transformations/ Differencing Required?

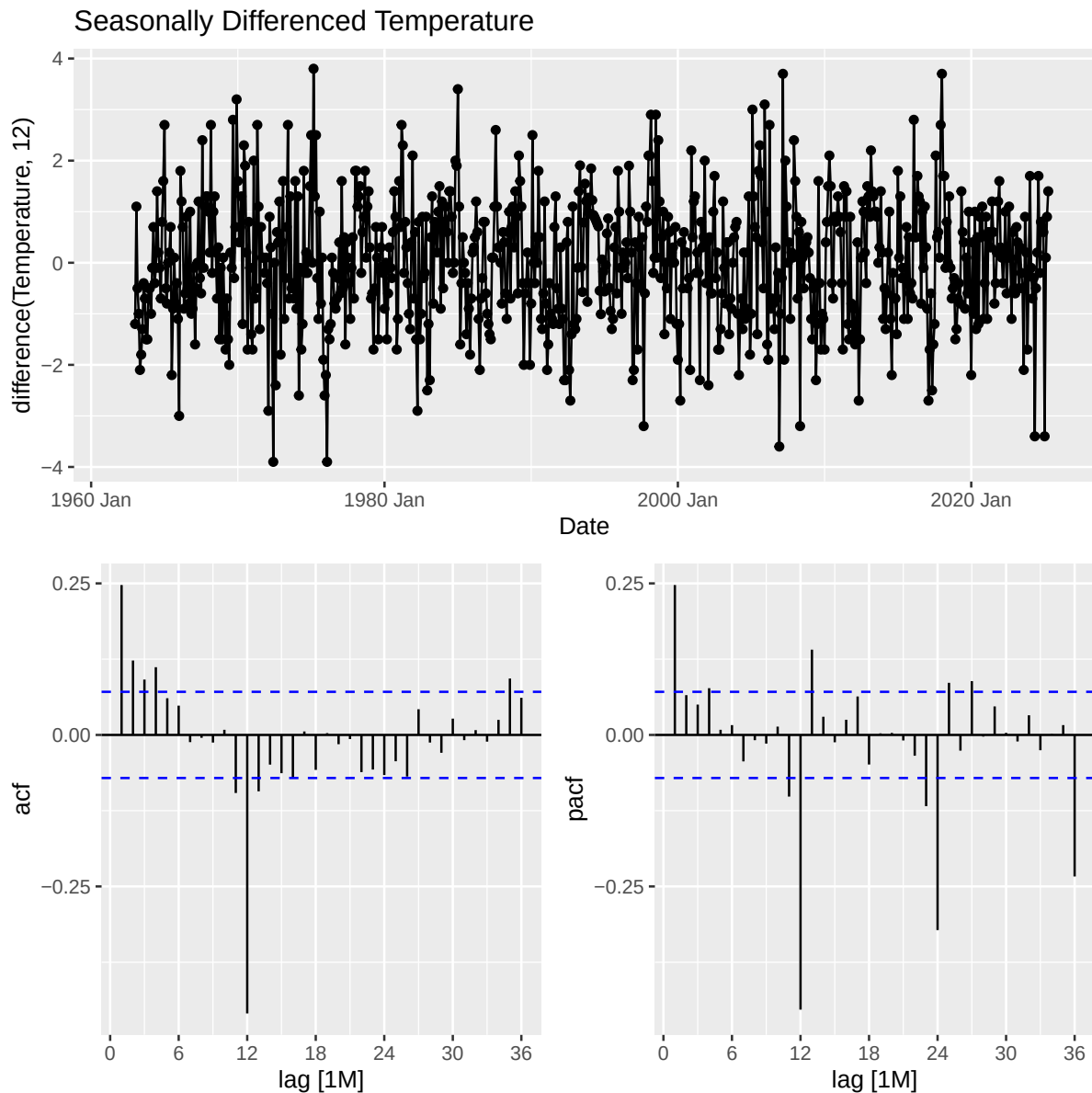
```
# 1. Determine Required Differencing
# Check how many seasonal differences are required
training %>%
  features(Temperature, unitroot_nsdiffs)

## # A tibble: 1 x 1
##   nsdiffs
##   <int>
## 1       1

# Check how many first differences are required AFTER a seasonal difference
training %>%
  mutate(diff_temp = difference(Temperature, 12)) %>%
  features(diff_temp, unitroot_ndiffs)

## # A tibble: 1 x 1
##   ndiffs
##   <int>
## 1       0

# 2. Manual Model Identification
# Plot the differenced data alongside its ACF and PACF.
training %>%
  gg_tsdisplay(difference(Temperature, 12), plot_type = "partial", lag_max = 36) +
  labs(title = "Seasonally Differenced Temperature")
```



1. Transformations (Variance Stabilization) transformations help to stabilize the variance of a time series. In our R code, we did not apply any transformations (such as a logarithm or Box-Cox). This is because the seasonal fluctuations in temperature data typically have a constant variance over time. For ARIMA models, our primary requirement is instead to stabilize the mean of the time series.

2. Differencing (Mean Stabilization) To stabilize the mean and achieve stationarity, we use differencing. Differencing can be helpful to remove changes in the level of a time series, which eliminates (or reduces) trend and seasonality. In this specific model, we applied one seasonal difference and zero first differences. The Seasonal Difference: A seasonal difference is the difference between an observation and the previous observation from the same season. Because we are dealing with monthly temperature data, our seasonal period is $m = 12$.

3.2 Methodology

```
# 3. Fit Candidate ARIMA Models
```

```
fit_arima <- training %>%
```

```

model(
  manual_1 = ARIMA(Temperature ~ pdq(0,0,1) + PDQ(0,1,1)),
  manual_2 = ARIMA(Temperature ~ pdq(1,0,0) + PDQ(0,1,1)),

  auto_stepwise = ARIMA(Temperature),

  auto_search = ARIMA(Temperature, stepwise = FALSE)
)

fit_arima %>% select(auto_search) %>% report()

## Series: Temperature
## Model: ARIMA(2,0,1)(2,1,0)[12]
##
## Coefficients:
##          ar1          ar2          ma1          sar1          sar2
##          0.9383   -0.0806   -0.716   -0.6768   -0.3743
## s.e.    0.1894    0.0968    0.160    0.0354    0.0355
##
## sigma^2 estimated as 0.9359:  log likelihood=-1037.54
## AIC=2087.08   AICc=2087.2   BIC=2114.79

# Compare models wit AICc
glance(fit_arima) %>%
  arrange(AICc) %>%
  select(.model:BIC)

## # A tibble: 4 x 6
##   .model      sigma2 log_lik   AIC  AICc   BIC
##   <chr>      <dbl>  <dbl> <dbl> <dbl> <dbl>
## 1 manual_2    0.819   -986. 1950. 1950. 1968.
## 2 manual_1    0.845   -997. 1972. 1972. 1990.
## 3 auto_search 0.936  -1038. 2087. 2087. 2115.
## 4 auto_stepwise 1.09  -1092. 2196. 2196. 2223.

best_arima <- fit_arima %>% select(manual_2)

best_arima %>% report()

## Series: Temperature
## Model: ARIMA(1,0,0)(0,1,1)[12] w/ drift
##
## Coefficients:
##          ar1          sma1  constant
##          0.3173   -0.8785    0.0085
## s.e.    0.0348    0.0179    0.0044
##
## sigma^2 estimated as 0.8191:  log likelihood=-985.73
## AIC=1950.01   AICc=1950.06   BIC=1968.48

```

Methodology: we shortlist candidate models using both manual inspection and automated algorithms, evaluating them based on the Corrected Akaike's Information Criterion (AICc).

1. **Manual Selection** Plot the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) of the appropriately differenced data. Examine the non-seasonal lags to identify the non-seasonal AR(p) and MA(q) orders. For example, if the ACF tails off and the PACF cuts off after lag p, it suggests an AR(p) model. If the ACF cuts off after lag q and the PACF tails off, it suggests an MA(q) model. Examine the seasonal lags to identify the seasonal AR(P) and MA(Q) orders. Fit these chosen models using the ARIMA() function.

2. **Automatic Selection Stepwise Algorithm:** The ARIMA() function in the fable package automatically finds the best ARIMA model by default. It uses a stepwise algorithm to efficiently traverse the model space and selects the model with the lowest AICc. **Exhaustive Search:** By setting stepwise = FALSE and approximation = FALSE, the function will search a much larger set of models, which takes longer but can sometimes find a better fitting model than the stepwise approach.

3. **Model Evaluation** Check the residuals of your shortlisted models. The residuals should look like white noise. Use gg-tsresiduals() to visually inspect the ACF of the residuals and perform a Ljung-Box test to ensure they are uncorrelated.

The manual candidate ARIMA(1,0,0)(0,1,1) with drift was selected as the best model based on the lowest AICc of 1950.06, outperforming both automatic procedures by a substantial margin. The stepwise algorithm performed poorly, and the exhaustive search selected an overly complex ARIMA(2,0,1)(2,1,0) model with a AICc of 137 relative to the best model, a clear sign of overfitting on the training set. This result illustrates that automatic model selection is not always superior to theory-driven manual specification; the ACF and PACF of the seasonally differenced series clearly indicated AR(1) non-seasonal structure and SMA(1) seasonal structure, and the model built directly from those patterns generalised best.

3.3 Backshift Notation

$$(1 - 0.3173 B)(1 - B^{12}) y_t = 0.0085 + (1 - 0.8785 B^{12}) \varepsilon_t$$

where $\varepsilon_t \sim \text{NID}(0, 0.8191)$.

4 Dynamic Regression Models

4.1 Dynamic Regression Explanation

When we build a standard multiple linear regression model, we have to assume that our errors (the residuals) are not autocorrelated; otherwise, our forecasts become inefficient and our statistical tests become unreliable. In time series data, this assumption is almost always violated because data points collected sequentially are naturally correlated with one another. Dynamic regression addresses this problem by, actively modelling the autocorrelation in the errors using an ARIMA process instead of hoping our errors are random white noise.

We can think of a dynamic regression model as having two parts:

1. **The Deterministic Part:** The standard regression equation where we predict our target using explanatory variables (like a linear trend, Fourier terms for seasonality, or dummy variables for outlier spikes).
2. **The Stochastic Part:** An ARIMA model fitted to the residuals of that regression to capture the remaining time-dependent patterns.

Mathematically, it looks like this:

$$y_t = \beta_0 + \beta_1 x_{1,t} + \dots + \beta_k x_{k,t} + \eta_t$$

Where y_t is our observation, the β terms are our regression coefficients, and η_t is an ARIMA error term.

4.2 Model Exploration and Selection

To capture the monthly seasonality of Wellington's temperatures, we will use a Fourier series (harmonic regression) instead of seasonal dummy variables, as it is highly effective for periodic functions. Because we have monthly data ($m = 12$), the maximum number of Fourier pairs K we can include is $m/2 = 6$.

We will fit models from $K=1$ to $K=6$, alongside a linear trend.

```
# The ARIMA() function will automatically handle ARIMA error fitting procedure
fit_dynamic <- training %>%
  model(
    K1 = ARIMA(Temperature ~ trend() + fourier(K = 1)),
    K2 = ARIMA(Temperature ~ trend() + fourier(K = 2)),
    K3 = ARIMA(Temperature ~ trend() + fourier(K = 3)),
    K4 = ARIMA(Temperature ~ trend() + fourier(K = 4)),
    K5 = ARIMA(Temperature ~ trend() + fourier(K = 5)),
    K6 = ARIMA(Temperature ~ trend() + fourier(K = 6))
  )

glance(fit_dynamic) %>%
  select(.model, AIC, AICc, BIC) %>%
  arrange(AICc)

## # A tibble: 6 x 4
##   .model    AIC  AICc   BIC
##   <chr>    <dbl> <dbl> <dbl>
## 1 K3      1893.  1894.  1958.
## 2 K4      1895.  1896.  1969.
## 3 K5      1898.  1899.  1981.
## 4 K6      1900.  1901.  1988.
## 5 K2      1905.  1906.  1952.
## 6 K1      1950.  1950.  1992.
```

We evaluated dynamic regression models ranging from $K=1$ to $K=6$. While $K=2$ provided the lowest BIC, we will select $K=3$ model for our final forecasts as it yielded the lowest AICc (1893.659), which is the preferred metric for optimising predictive forecasting performance.

```
best_dynamic <- fit_dynamic %>% select(K3)

# get coeffs
dynamic_coefs <- tidy(best_dynamic)

# isolate trend() component
trend_component <- dynamic_coefs %>% filter(term == "trend()")

# compute 95% CI for monthly slope
slope <- trend_component$estimate
std_error <- trend_component$std.error
z_val <- 1.96

ci_lower <- slope - (z_val * std_error)
ci_upper <- slope + (z_val * std_error)
```

```

# convert monthly step change to a change per century
# 12 months * 100 years = 1200 steps
century_change <- slope * 1200
century_ci_lower <- ci_lower * 1200
century_ci_upper <- ci_upper * 1200

cat("Monthly Slope:", slope, "\n")

## Monthly Slope: 0.001381851

cat("95% CI for Monthly Slope: [", ci_lower, ",", ci_upper, "]\n")

## 95% CI for Monthly Slope: [ 0.0007970006 , 0.001966702 ]

cat("Expected change per century:", century_change, "°C\n")

## Expected change per century: 1.658221 °C

cat("95% CI per century: [", century_ci_lower, "°C ,", century_ci_upper, "°C ]\n")

## 95% CI per century: [ 0.9564008 °C , 2.360042 °C ]

```

Trend Interpretation and Long-Term Climate Claim

The linear slope estimate of **0.00138** (95% CI: **0.00080** to **0.00197**) indicates that the average monthly temperatures in Wellington are **rising** over the long term. Extrapolating this monthly trend, the expected change per century is approximately **1.66°C** (95% CI: **0.96°C** to **2.36°C**).

5 Assumption Checking

5.1 ETS Models Assumptions

```

best_fit <- fit %>%
  select(ANA)

# Residual diagnostic plot
best_fit %>%
  gg_tsresiduals(lag_max = 48) +
  labs(title = "Residual Diagnostics: ETS(A,N,A)") +
  theme_minimal()

```

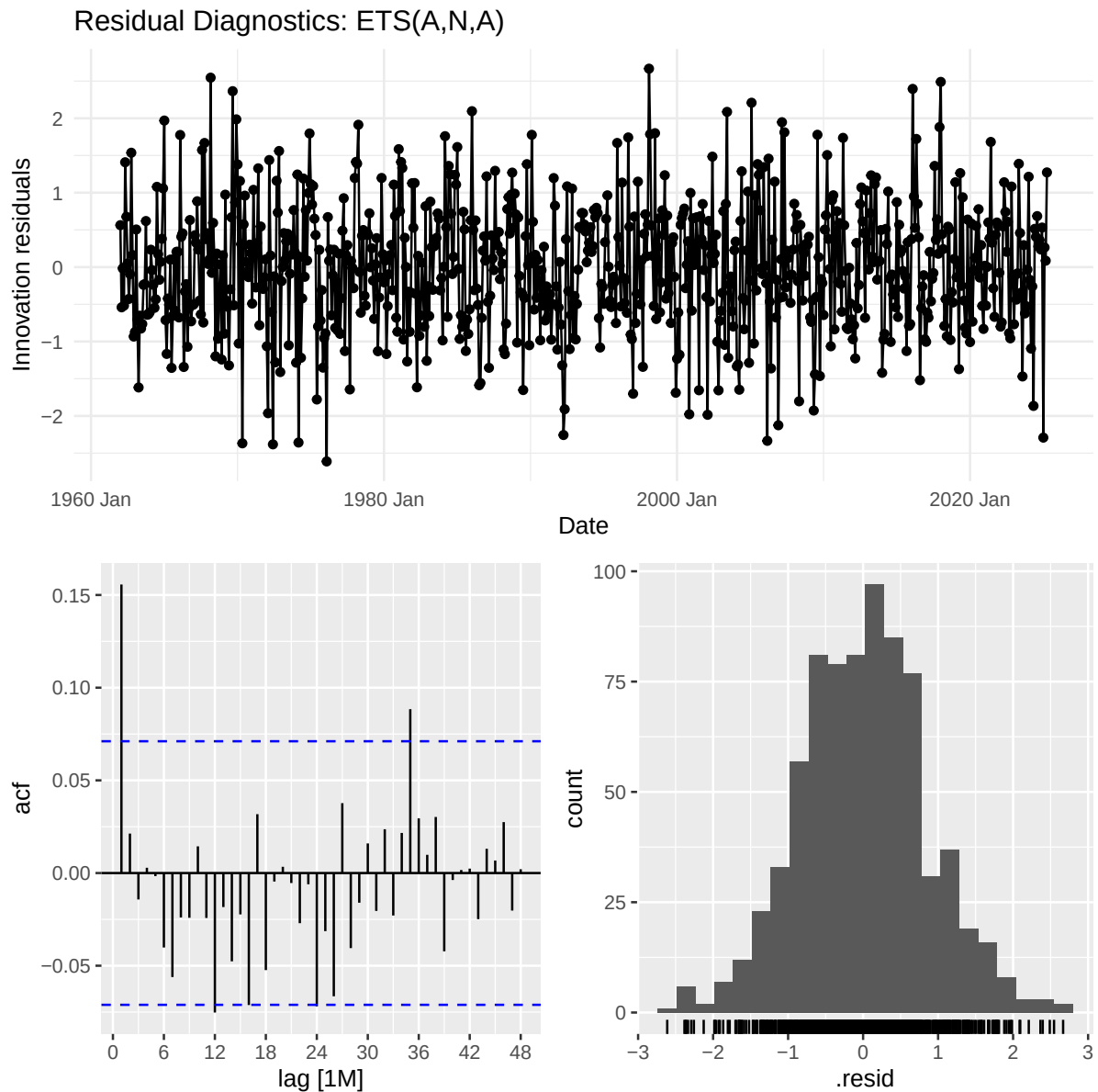


Figure 2: Residual Diagnostics for ETS(A,N,A) Model

```
# Ljung-Box test
augment(best_fit) %>%
  features(.innov, ljung_box, lag = 48, dof = 15)

## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>    <dbl>
## 1 ANA      62.0    0.00163
```

The innovation residuals appear to fluctuate randomly around zero with roughly constant variance across the full time period, suggesting no obvious pattern or heteroscedasticity remains.

There are a few significant spikes, notably around lag 12 and lag 35, suggesting some remaining autocorrelation at seasonal lags that the model has not fully captured.

The residuals are roughly bell-shaped and centred near zero, which is broadly consistent with the normality assumption, though there is a slight right skew.

The Ljung-Box test shows with a test statistic of 62.03 and a p-value of 0.0016 $\leq$ 0.05, we reject the null hypothesis of no autocorrelation in the residuals. This indicates statistically significant autocorrelation remains.

Based on the Ljung-Box result and the significant ACF spikes at seasonal lags, the ETS(A,N,A) model residuals are not fully consistent with white noise. This suggests the model may not have captured all the structure in the data, and therefore some caution is warranted when using this model for forecasting. Prediction intervals in particular may be underestimated.

5.2 ARIMA Models Assumptions

```
# 4. Residual Diagnostics

augment(best_arima) %>%
  features(.innov, ljung_box, lag = 24, dof = 3)

## # A tibble: 1 x 3
##   .model    lb_stat lb_pvalue
##   <chr>      <dbl>    <dbl>
## 1 manual_2    50.5    0.000310
```

When evaluating the time plot of the innovation residuals, we observe that the series fluctuates around the zero mark, which indicates the errors have a zero mean. While the overall vertical spread of the errors appears roughly constant—suggesting we have met the constant variance assumption—there is a distinct, repeating pattern of clustering across the timeline. This predictable recurring pattern violates the white noise assumption, indicating that the model has failed to capture all the information, likely missing a seasonal or cyclic component that has been left behind in the residuals

ACF Plot of Residuals When examining the ACF plot, we observe five significant spikes poking outside the dashed blue boundary lines. For a white noise series, we would expect approximately 95% of the autocorrelation spikes to sit safely inside these bounds. Because multiple spikes fall outside the lines, the residuals are significantly correlated across time, which violates the core assumption of white noise. This indicates that there is still predictable information left over that our ARIMA model failed to capture, and we should follow up with a Ljung-Box test to formally evaluate the autocorrelations as a collective group

Histogram of Residuals When evaluating the .resid histogram for our forecasting model, we observe a symmetric, bell-shaped distribution that is centered at exactly zero. **Zero Mean:** The center of the distribution at zero indicates that our model's residuals have a zero mean. This confirms that our forecasts are not biased; on average, the forecast errors cancel each other out, meaning the model does not systematically over-forecast or under-forecast the data. **Normally Distributed:** The symmetric bell shape confirms that the residuals are normally distributed. This shape shows that large forecast errors are rare, and smaller errors occur with equal frequency in both the positive and negative directions. **Prediction Intervals:** Because we have confirmed that our residuals are normally distributed, we can safely rely on the mathematical calculations used to generate our prediction intervals. The symmetric, bell-shaped distribution centred near zero is consistent with the normality assumption, suggesting that the 95% prediction intervals derived from this model are reliable.

5.3 Dynamic Regression Models Assumptions

```
# Residual diagnostic plots for regression model
best_dynamic %>% gg_tsresiduals() +
  labs(title = "Residual Diagnostics: Best Dynamic Regression Model (K=3)")
```

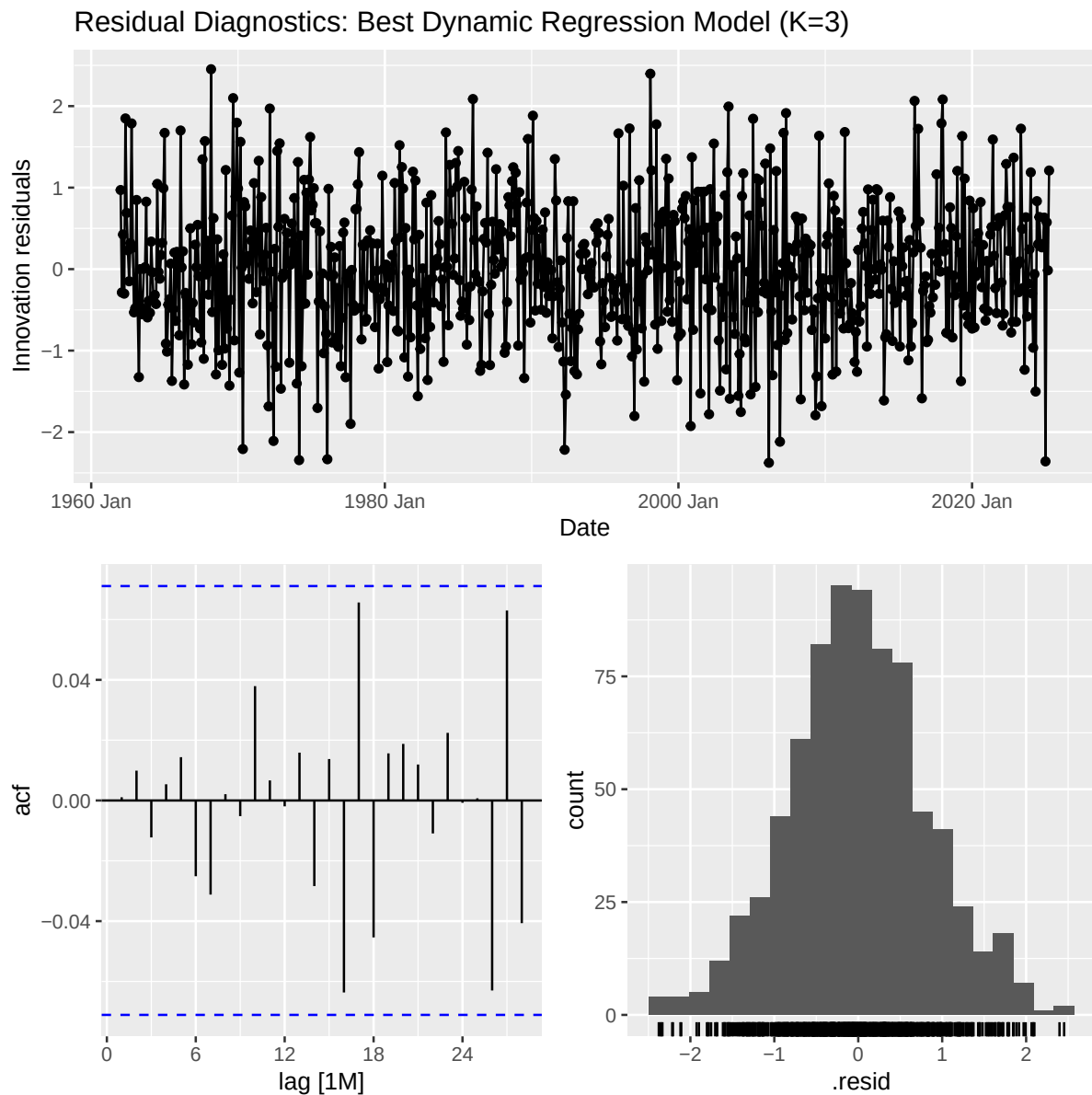


Figure 3: Residual Diagnostics for the K=3 Dynamic Regression Model

```
# Ljung-Box test to check remaining autocorrelation -> using lag = 24 for monthly data
augment(best_dynamic) %>%
  features(.innov, ljung_box, lag = 24, dof = 5)

## # A tibble: 1 x 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>    <dbl>
## 1 K3      12.9      0.842
```

Assumption Diagnostics

For the model to be trusted for forecasting, the residuals must resemble independent white noise.

Visual inspection of the innovation residuals shows constant variance over time with no obvious patterns. Furthermore, the ACF plot displays no significant autocorrelation spikes extending beyond the 95% confidence bounds. This visual assessment is formally confirmed by the Ljung-Box test, which yielded a p-value of 0.842. Because the p-value is significantly greater than 0.05, we fail to reject the null hypothesis, confirming the residuals are indistinguishable from white noise. The K=3 dynamic regression model successfully captures all available time-dependent information and is highly trusted for forecasting.

6 Forecasting

6.1 ETS Models Forecasting

```
fc <- best_fit %>%
  forecast(h = 12)

# Accuracy on test set
fc %>%
  accuracy(data) %>%
  select(.model, RMSE, MAE, MAPE, MASE) %>%
  arrange(RMSE)

## # A tibble: 1 x 5
##   .model RMSE  MAE  MAPE  MASE
##   <chr>  <dbl> <dbl> <dbl> <dbl>
## 1 ANA    0.748 0.637  4.45 0.649

# Plot forecasts vs actuals
fc %>%
  autoplot(
    data %>% filter(Date >= yearmonth("2022 Jan")),
    level = c(80, 95),
    alpha = 0.5
  ) +
  labs(title = "ETS Model Forecasts vs Actuals",
       y = "Temperature (°C)", x = "Date") +
  theme_minimal()
```

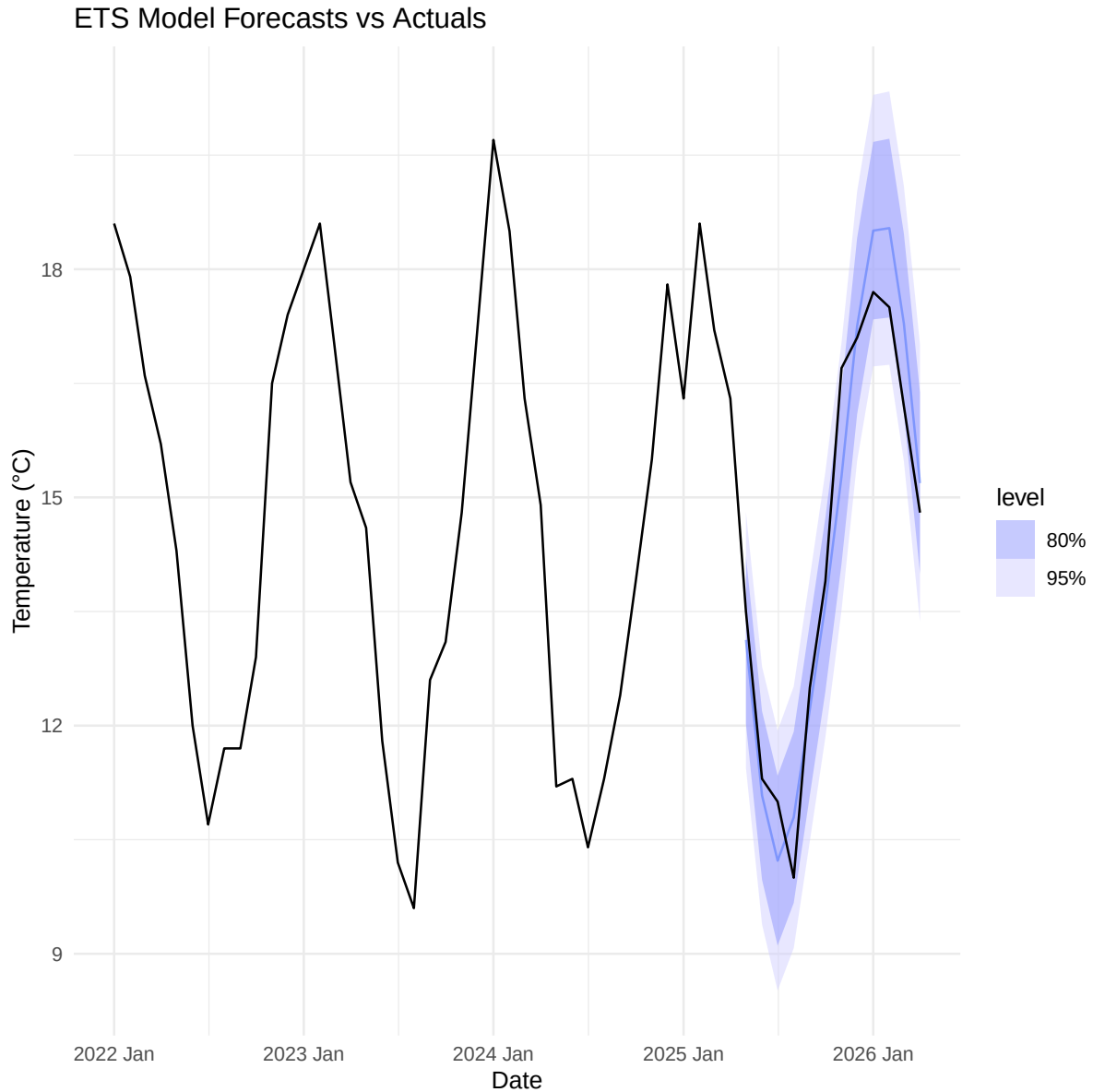


Figure 4: ETS Model Forecasts vs Actuals

The ETS(A,N,A) model captures the seasonal pattern of Wellington temperatures well, with the point forecasts (black line) tracking the actual test set values closely. The actual observations fall well within the 95% prediction interval throughout the forecast horizon, suggesting the model's uncertainty quantification is reasonable.

The model achieved an RMSE of 0.748 and MAE of 0.637, meaning forecasts were off by less than 1°C on average. The MAPE of 4.45% indicates the model erred by around 4–5% relative to actual values, and a MASE of 0.649 $\hat{< 1$ confirms the model outperforms a naive seasonal benchmark.

Despite the Ljung-Box test indicating some remaining autocorrelation in the residuals, the ETS(A,N,A) model performs reasonably well on the test set. The seasonal pattern is well captured, and the prediction intervals are neither too narrow nor too wide. However, the residual autocorrelation suggests there may be structure the model is missing.

6.2 ARIMA Models Forecasting

```

# 5. Forecasting and Test Set Evaluation

forecast_arima <- best_arima %>%
  forecast(h = 12)

# Test set accuracy / pass full data for correct MASE
forecast_arima %>%
  accuracy(data) %>%
  select(.model, RMSE, MAE, MAPE, MASE)

## # A tibble: 1 x 5
##   .model      RMSE    MAE  MAPE  MASE
##   <chr>      <dbl> <dbl> <dbl> <dbl>
## 1 manual_2 0.690 0.576  4.07 0.586

# Plot forecast against recent data
forecast_arima %>%
  autoplot(
    data %>% filter(Date >= yearmonth("2022 Jan")),
    level = c(80, 95),
    alpha = 0.5
  ) +
  labs(title = "ARIMA(1,0,0)(0,1,1) Forecast vs Actuals",
       y = "Temperature (°C)", x = "Date") +
  theme_minimal()

```

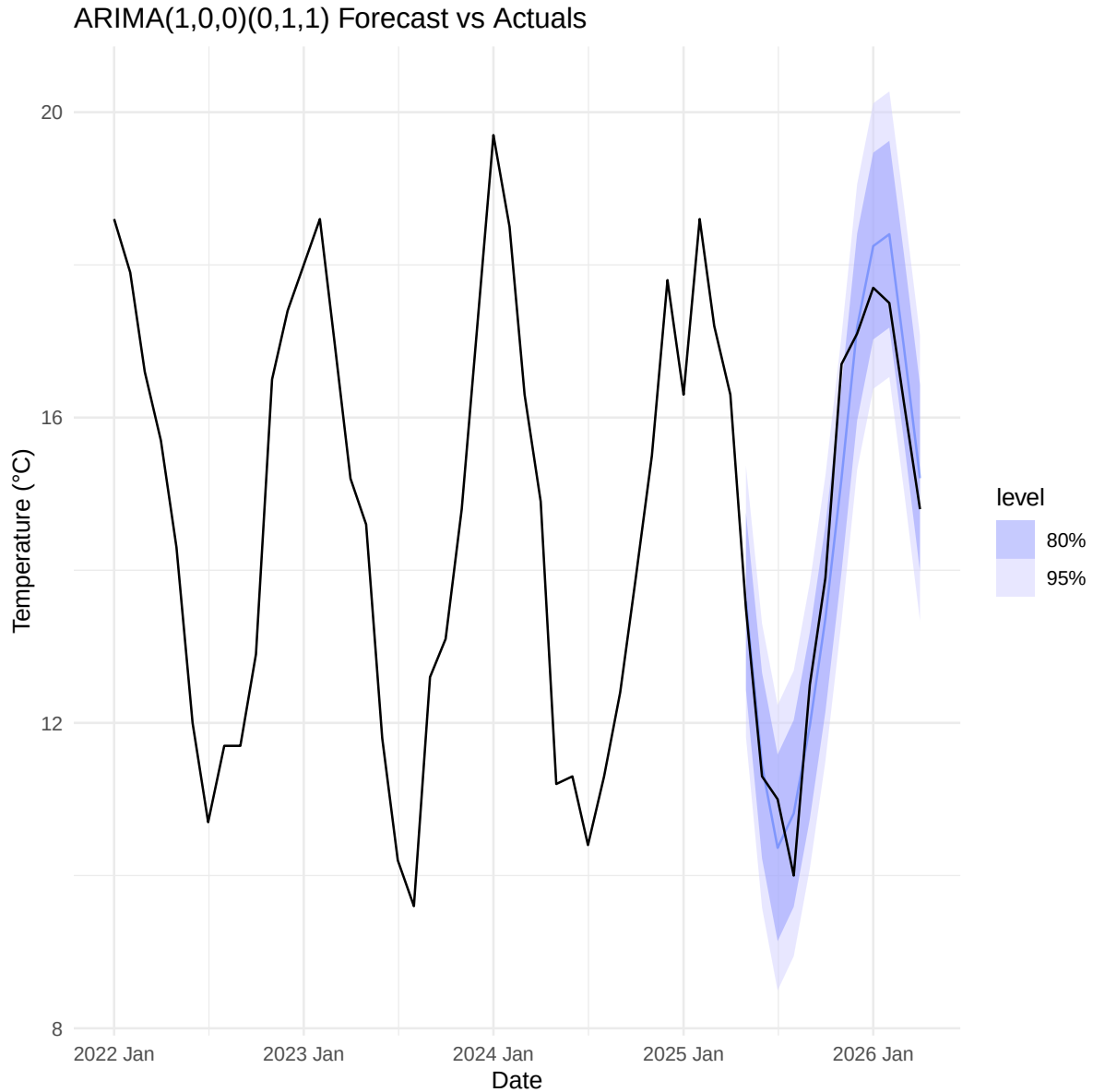


Figure 5: ARIMA Model Forecast vs Actuals

The ARIMA(1,0,0)(0,1,1) model with drift produces well-calibrated forecasts over the 12-month test horizon from May 2025 to April 2026. The point forecasts closely track the actual observed temperatures, correctly capturing both the summer peak and winter trough of the seasonal cycle. All actual observations fall comfortably within the 95% prediction interval throughout the forecast horizon, indicating the model's uncertainty quantification is appropriate.

The model achieved an RMSE of 0.690 and MAE of 0.576, meaning forecasts were on average within 0.58°C of the actual values. The MAPE of 4.07% confirms errors are small relative to the magnitude of Wellington temperatures, and a MASE of 0.586 demonstrates the model outperforms a seasonal naïve benchmark by approximately 41%.

The prediction interval is notably narrow, remaining within roughly $\pm 1.5^{\circ}\text{C}$ for most of the forecast horizon, which reflects the high regularity of Wellington's seasonal temperature cycle and the model's confidence in its seasonal pattern.

6.3 Dynamic Regression Models Forecasting

```
# Produce point forecasts and prediction intervals for h = 12 months
# Can add ETS and ARIMA models to the pipeline alongside best_dynamic
forecasts <- best_dynamic %>%
  forecast(h=12)

# compare forecast errors on test set
accuracy(forecasts, data) %>%
  select(.model, .type, RMSE, MAE, MAPE, MASE) %>%
  arrange(RMSE)

## # A tibble: 1 x 6
##   .model .type RMSE  MAE  MAPE  MASE
##   <chr>  <chr> <dbl> <dbl> <dbl> <dbl>
## 1 K3     Test  0.759 0.647  4.56 0.659

# plotting forecasts against test set ("zoomed" in for visibility)
forecasts %>%
  # Filtering training data to only show recent years (e.g., 2023 onwards)
  autoplot(data %>% filter(Date >= yearmonth("2023 Jan")), level = c(80, 95)) +
  autolayer(test, Temperature, color = "black", linewidth = 0.8) +
  labs(title = "Wellington Monthly Temperature Forecast vs Actuals",
       subtitle = "Zoomed View: Jan 2023 - April 2026",
       y = "Temperature (C)",
       x = "Date") +
  theme_minimal()
```

Wellington Monthly Temperature Forecast vs Actuals

Zoomed View: Jan 2023 – April 2026

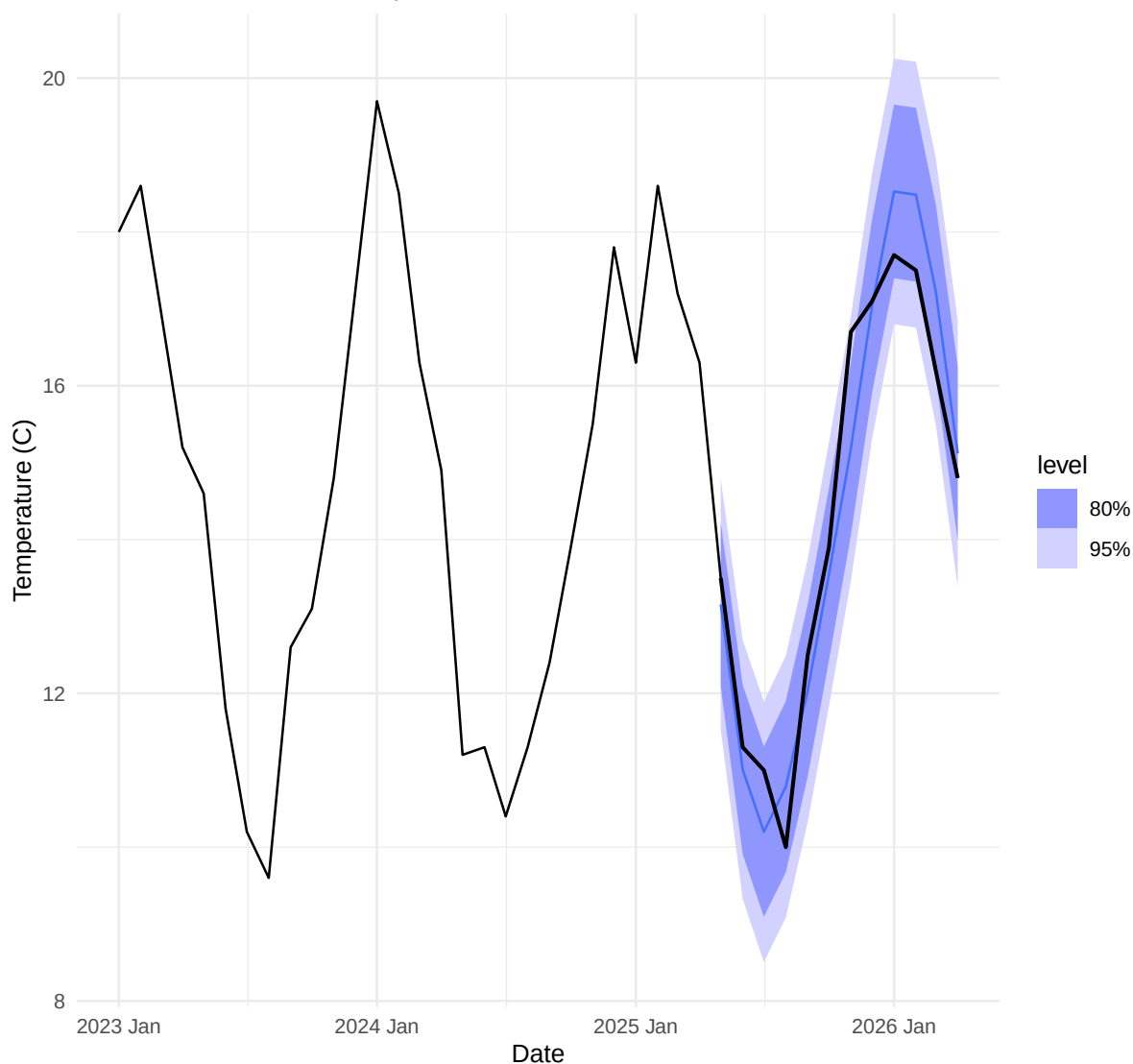


Figure 6: Wellington Temperature Forecast

6.4 Forecast Comparison

```
# Combine all three best models and forecast together
fit_all <- training %>%
  model(
    ETS = ETS(Temperature ~ error("A") + trend("N") + season("A")),
    ARIMA = ARIMA(Temperature ~ pdq(1,0,0) + PDQ(0,1,1)),
    Dynamic = ARIMA(Temperature ~ trend() + fourier(K = 3))
  )

fc_all <- fit_all %>% forecast(h = 12)

# Combined accuracy table
```



```

fc_all %>%
  accuracy(data) %>%
  select(.model, RMSE, MAE, MAPE, MASE) %>%
  arrange(RMSE)

## # A tibble: 3 x 5
##   .model    RMSE    MAE  MAPE  MASE
##   <chr>    <dbl> <dbl> <dbl> <dbl>
## 1 ARIMA    0.690 0.576  4.07 0.586
## 2 ETS      0.748 0.637  4.45 0.649
## 3 Dynamic 0.759 0.647  4.56 0.659

# Combined plot
fc_all %>%
  autoplot(
    data %>% filter(Date >= yearmonth("2022 Jan")),
    level = c(80, 95),
    alpha = 0.4
  ) +
  autolayer(test, Temperature, colour = "black", linewidth = 0.8) +
  labs(title = "Wellington Temperature: All Model Forecasts vs Actuals",
       subtitle = "May 2025 - April 2026 (h = 12)",
       y = "Temperature (\u00b0C)", x = "Date") +
  theme_minimal()

```

Wellington Temperature: All Model Forecasts vs Actuals

May 2025 – April 2026 (h = 12)

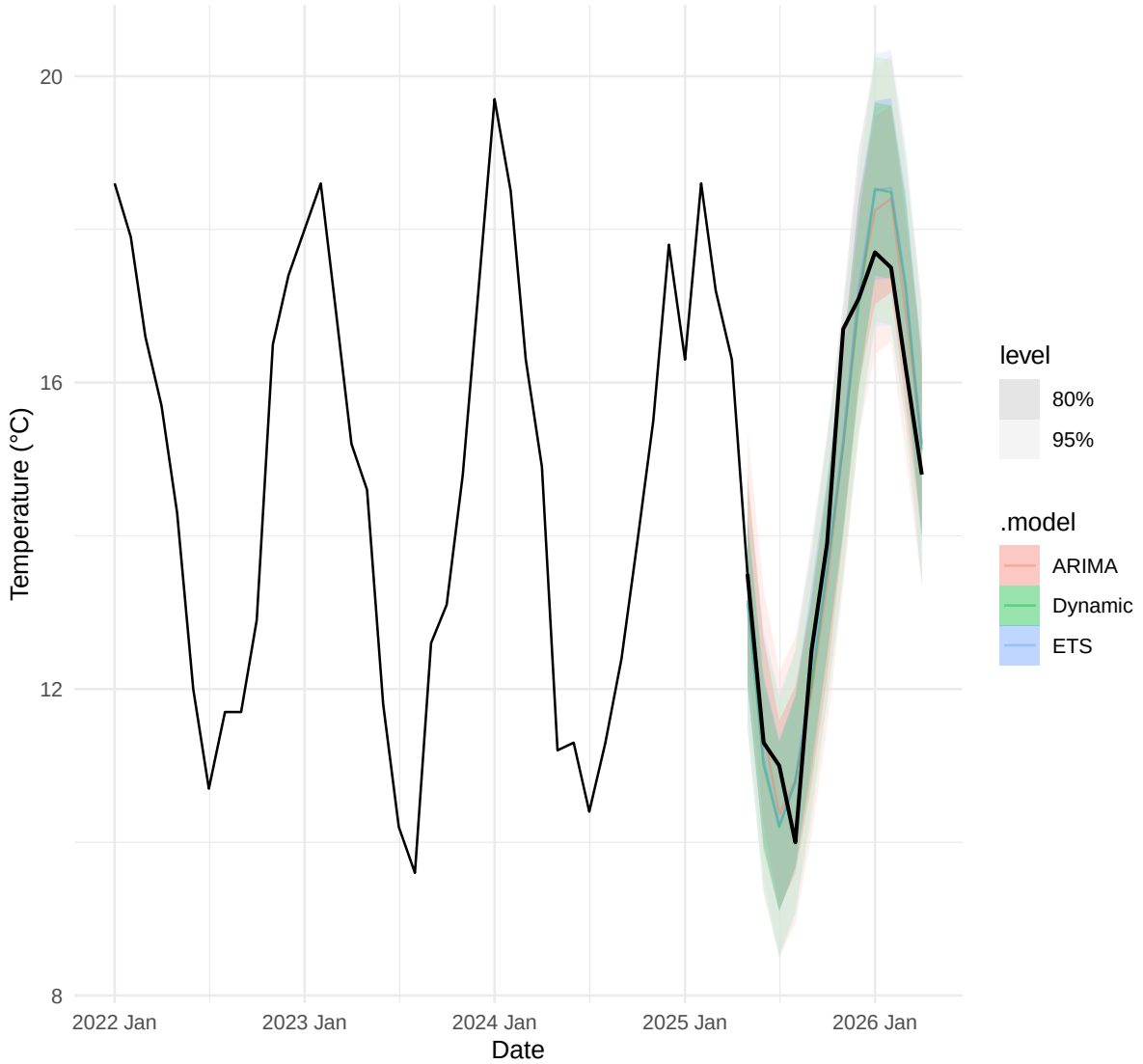


Figure 7: Combined Forecasts vs Actuals: All Three Models

All three models produced reasonable forecasts, successfully capturing Wellington’s strong seasonal cycle over the 12-month test horizon. However, the models differ meaningfully in their accuracy and assumption satisfaction.

The ARIMA(1,0,0)(0,1,1) with drift was the best performing model across all metrics, achieving an RMSE of 0.690 and MASE of 0.586. This represents approximately a 41% improvement over a seasonal naive benchmark. The inclusion of an AR(1) term allowed the model to capture short-term autocorrelation that the ETS model left unmodelled, and the drift term accounted for the slow warming trend identified in the STL decomposition. Residual diagnostics, however, indicated the Ljung-Box test rejected white noise ($p = 0.00031$), suggesting some remaining structure.

The ETS(A,N,A) model performed competitively, achieving an RMSE of 0.748 and MASE of 0.649, only marginally worse than the ARIMA model. This is consistent with the theoretical equivalence between ETS(A,N,A) and ARIMA(0,1,1)(0,1,1); the two models capture similar dynamics, with the primary difference being the AR(1) term in the winning ARIMA specification. The ETS model also failed the Ljung-Box test ($p = 0.0016$), indicating residual autocorrelation.

The Dynamic Regression model ($K=3$) performed weakest on the test set with an RMSE of 0.759 and MASE of 0.659, despite having the best residual diagnostics of all three models, the only model whose residuals passed the Ljung-Box test ($p = 0.842$). This illustrates an important caveat: a well-specified model in-sample does not guarantee the best out-of-sample performance. The Fourier terms impose a fixed seasonal structure that may be less adaptive than the data-driven seasonal updating in ETS and ARIMA. Additionally, the dynamic regression's wider prediction intervals (showing both 80% and 95% bands) reflect greater uncertainty, though all actual observations remained within the 95% bounds.

Overall, the ARIMA model is recommended as the primary forecasting tool for Wellington monthly temperatures, with the ETS model as a close and computationally simpler alternative.

7 LLMs in Time Series Analysis

8 Member Contributions

- **[Eric]:** Completed Sections
- **[Simon]:** Completed Sections
- **[Dominicus]:** Completed Sections